



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

LABORATORIO 3

Automatización, Modelamiento y Orquestación de Procesos con n8n: Sistema de Devoluciones de AndesHogar SpA

Departamento de Industrias

ICN-292 — Sistemas de Información para la Gestión

Paralelo: 101

Profesor: J. Saez

Alumno: Carlos Martínez González

Rol: 202466517-0

URL Repositorio: <https://github.com/emg46408-cmd/ICN292-Lab3-Martinez-Carlos>



Resumen ejecutivo

Se construyó en n8n un producto mínimo viable de triage de devoluciones para AndesHogar SpA, compuesto por tres workflows: (1) triage, que recibe cada solicitud por Webhook POST, la clasifica con un Switch en modo Rules (sin código) según la política y mis parámetros $U = 74.000$ y $D = 7$, consulta la UF en mindicator.cl, notifica al cliente, registra la trazabilidad (id, SKU, monto, ruta, motivo, U y D) y responde al emisor; (2) emisor, que envía las 15 solicitudes y los casos construidos al webhook; y (3) resumen, programado con Schedule Trigger, que consolida el día con Summarize y envía un único mensaje.

Principales conclusiones.

- Las reglas se aplican en el orden de la política y las comparaciones son inclusivas: una solicitud con monto = 74.000 o con 7 días desde la compra queda aprobada.
- El flujo maneja bien los datos inválidos previstos, pero no puede resolver dos casos: un monto no numérico ("veinte mil") hace fallar el Switch y un SKU inexistente ("ZZ-999") se clasifica como válido.
- Un flujo programado sin Error Workflow falla en silencio; el resumen diario debe tener uno asociado y un tratamiento explícito del día sin solicitudes.

Recomiendo convertir el MVP en el sistema definitivo, condicionado a cerrar cinco brechas antes de operar. A favor: el triage es determinista, por lo que automatizarlo ataca el verdadero cuello de botella —el tiempo de primera respuesta de tres días hábiles frente a las 24 horas prometidas— sin tocar la inspección física, que sigue siendo manual; además cada registro es auditable (ruta, motivo, U y D) y la política puede cambiarse sin programar. En contra: La Parte C mostró que hoy el MVP no es robusto. Condiciones: (1) validar tipo y formato de los campos antes del Switch; (2) validar el SKU contra el catálogo; (3) asociar un Error Workflow con alertas a todos los flujos, en especial al programado; (4) reemplazar Google Sheets por una base de datos y autenticar el webhook; (5) mantener U y D como parámetros configurables y ejecutar una batería de regresión (15 solicitudes + 2 casos de frontera) ante cada cambio de política.

1. Contexto y alcance

El canal e-commerce de AndesHogar SpA recibe las solicitudes de devolución por un formulario web y hoy las procesa a mano el equipo de Servicio al Cliente: el tiempo promedio de primera respuesta es de tres días hábiles, aunque la política publicada promete 24 horas. Tras modelar el proceso en BPMN (Laboratorio 2), se identificó que el cuello de botella no es la inspección física del producto, que necesariamente es manual, sino el triage: decidir para cada solicitud si se aprueba de inmediato, si requiere revisión en bodega o si se rechaza.

Este laboratorio construye en n8n un MVP que recibe las solicitudes, decide la ruta, registra el resultado y responde, y entrega una recomendación fundada sobre su conversión en sistema definitivo (ver resumen ejecutivo). Todos los campos de entrada llegan como texto, tal como los enviaría un formulario web.

2. Declaración de parámetros personales

Parámetro	Valor
Semilla S	944

Umbral U	74.000
Plazo D	7 días

3. Parte A — Flujo de triage

3.1 Arquitectura del workflow

#	Nodo	Tipo / configuración	Función
1	Webhook	Método POST; path propio [indicar path]	Recibe cada solicitud como JSON con campos de texto.
2	Switch	Modo Rules; 6 reglas + Fallback Output	Aplica la política en orden de precedencia
3–6	Edit Fields ×4	Set (Manual Mapping)	Escriben ruta y motivo en cada rama
7	HTTP Request	GET a mindicador.cl	Obtiene la UF vigente para expresar el monto en UF.
8	Notificación	Send Email (o Gmail)	Avisa al cliente (email_cliente) la ruta y el motivo.
9	Registro	Google Sheets Append row	Guarda la trazabilidad exigida por la regla 7.
10	Respond to Webhook	JSON	Devuelve la confirmación al emisor.

3.2 Switch en modo Rules

#	Condición	Detalle	Salida
1	String · is empty	{{ \$json.body.id_solicitud }}	INVALIDO_CAMPO
2	Number · ≤ 0	{{ \$json.body.monto.toNumber() }}	INVALIDO_MONTO
3	Number · > D (7)	{{ \$json.body.dias_desde_compra.toNumber() }}	RECHAZO_PLAZO
4	String · is equal to	estado_producto = danado_por_uso	RECHAZO_ESTADO
5	Number · > U (74000)	{{ \$json.body.monto.toNumber() }}	REVISION_MONTO
6	String · is equal to	estado_producto = con_fallas	REVISION_FALLA
—	Fallback Output	Ninguna regla se cumple	APROBACION

3.3 Nodos Edit Fields

Las siete salidas del Switch llegan a cuatro nodos Edit Fields, con la opción **Include Other Input Fields** activada para conservar los datos de la solicitud. Cuando dos causales comparten nodo, el motivo se fija con una expresión que distingue la causal, de modo que ningún registro queda sin motivo.

Nodo	Salidas que recibe	ruta	motivo
Edit Fields 1	INVALIDO_CAMPO, INVALIDO_MONTO	DATOS_INVALIDOS	"Falta id_solicitud" / "El monto no es mayor que cero"
Edit Fields 2	RECHAZO_PLAZO, RECHAZO_ESTADO	RECHAZO	"Plazo excedido: días desde la compra > D (7)" / "Producto dañado por uso"
Edit Fields 3	REVISION_MONTO, REVISION_FALLA	REVISION	"Monto > U (74.000): revisión en bodega" / "Producto con fallas: revisión en bodega"
Edit Fields 4	Fallback	APROBACION	"Cumple la política: aprobación automática"

3.4 HTTP Request (servicio público sin credenciales)

- URL: <https://mindicador.cl/api> (método GET, sin autenticación).
- Campo que consume: uf.valor, el valor de la UF vigente en pesos chilenos.
- Uso: monto_uf = monto / uf.valor, redondeado a dos decimales, para expresar el monto de la devolución en UF en el registro y en la respuesta.

3.5 Notificación y registro

La notificación se envía al correo del cliente (email_cliente) con el número de solicitud, la ruta y el motivo. El registro agrega una fila en Google Sheets con los campos exigidos (identificador, SKU, monto, ruta, motivo, U y D) y datos de apoyo:

Columna del registro	Origen
id_solicitud, sku, monto	Cuerpo del webhook
dias_desde_compra, estado_producto, email_cliente	Cuerpo del webhook
ruta, motivo	Edit Fields de la rama correspondiente
U_aplicado = 74000, D_aplicado = 7	Valores fijos de mis parámetros (S = 944)
uf_valor, monto_uf	HTTP Request a mindicador.cl
fecha_hora	{{ \$now }}

3.6 Workflow emisor

Como la barra del navegador solo envía GET, se construyó un segundo workflow: Trigger manually → nodo con las solicitudes (JSON) → HTTP Request (POST) a la URL del webhook, con el cuerpo

en JSON y los campos como texto. Envía las 15 solicitudes entregadas, los dos casos de frontera y los casos de la Parte C, uno por ítem.

3.7 URL de prueba y URL de producción del webhook

	URL de prueba (Test)	URL de producción (Production)
Ruta	/webhook-test/<path>	/webhook/<path>
Cuándo escucha	Solo tras pulsar <i>*Listen for test event*</i> / <i>*Execute workflow*</i> ; atiende una llamada	Siempre, mientras el workflow esté activo (publicado)
Datos	Se ven en el editor (nodo, entrada y salida)	Quedan en el historial de <i>*Executions*</i>

4. Parte B — Flujo programado de resumen

4.1 Diseño

#	Nodo	Configuración
1	Schedule Trigger	Diario a las 18:00, zona horaria America/Santiago.
2	Google Sheets · Get rows	Lee el registro del triage y filtra las filas con fecha_hora del.
3	Summarize	<i>*Group by*</i> ruta; Count de id_solicitud (solicitudes por ruta) y Sum de monto (monto total por ruta).

4.2 Frecuencia elegida

Se eligió una ejecución diaria a las 18:00. Cierra la jornada de Servicio al Cliente, es coherente con la promesa de respuesta en 24 horas (permite detectar al día siguiente cualquier acumulación de solicitudes en REVISION) y no genera ruido: una frecuencia horaria produciría mensajes casi vacíos y una semanal detectaría los problemas demasiado tarde. Como un e-commerce recibe solicitudes todos los días, el trigger no se limita a días hábiles.

4.3 Qué ocurre un día sin solicitudes

Si el registro no tiene filas del día, Google Sheets devuelve cero ítems y n8n no ejecuta los nodos siguientes: la ejecución termina como "exitosa" pero no se envía ningún mensaje, lo que es indistinguible de un flujo roto. Además, la tasa de aprobación sería una división $0 \div 0$. Por eso se activó **Always Output Data** en el nodo de lectura y se agregó un nodo IF que, ante cero solicitudes, envía el mensaje "Sin solicitudes hoy: 0 solicitudes, \$0, tasa de aprobación no aplicable". Así el operador recibe siempre el resumen, y su ausencia sí indica una falla.

5. Parte C — Errores, casos borde y límites

5.1 Tres entradas inválidas distintas

Caso 1 — Identificador vacío

Carga enviada

```
{
  "id_solicitud": "",
  "sku": "PL-120",
  "monto": "45990",
  "dias_desde_compra": "2",
  "estado_producto": "sin_abrir",
  "email_cliente": "cliente.inv1@example.cl"
}
```

Esperado

Regla 1 del Switch (*is empty*) → salida INVALIDO_CAMPO → Edit Fields 1: ruta = DATOS_INVALIDOS, motivo = "Falta id_solicitud". Se notifica, se registra y se responde.

Caso 2 — Monto negativo

Carga enviada

```
{
  "id_solicitud": "DEV-INV2",
  "sku": "PL-120",
  "monto": "-1500",
  "dias_desde_compra": "2",
  "estado_producto": "sin_abrir",
  "email_cliente": "cliente.inv2@example.cl"
}
```

Esperado

La regla 1 no se cumple (hay id); la regla 2 ($\text{monto.toNumber()} \leq 0$) sí → salida INVALIDO_MONTO → Edit Fields 1: ruta = DATOS_INVALIDOS, motivo = "El monto no es mayor que cero".

Caso 3 — Campo monto ausente (falta de nodo)

Carga enviada

```
{
  "id_solicitud": "DEV-INV3",
  "sku": "PL-120",
  "dias_desde_compra": "2",
  "estado_producto": "sin_abrir",
  "email_cliente": "cliente.inv3@example.cl"
}
```

Esperado

Según la política debería ser INVALIDO_MONTO. En el flujo, la regla 2 evalúa `undefined.toNumber()`: el Switch falla con error de expresión. Con `On Error = *Stop Workflow*` la ejecución se marca como fallida; no hay registro ni respuesta y, en producción, se dispara el Error Workflow. Es una brecha del MVP.

5.2 Los tres valores de On Error

Valor	Qué hace	Nodo(s)	Razón
Stop Workflow (por defecto)	Detiene la ejecución, la marca como fallida y, en producción, activa el Error Workflow.	Switch, Google Sheets , Respond to Webhook	Sin clasificación o sin registro no hay decisión auditable (regla 7): es preferible fallar a aprobar o descartar una solicitud sin dejar rastro.
Continue	Ignora el error y pasa el ítem de entrada por la salida normal.	Send Email	Un fallo del correo no debe impedir el registro; la fila en Sheets es la fuente de verdad. Se acepta que el error quede menos visible.
Continue (using error output)	Crea una segunda salida "Error" para tratar la falla en una rama propia.	HTTP Request (mindicador.cl)	La UF es un dato complementario: si la API cae, la rama de error fija monto_uf = "N/D" y el triage continúa sin bloquear al cliente.

5.3 Error Workflow y por qué un flujo programado falla en silencio

Un Error Workflow es un workflow separado que comienza con un nodo **Error Trigger** y que se asigna en **Settings → Error workflow** de otro workflow. Cuando una ejecución automática de este falla, n8n lanza el Error Workflow con datos como el nombre del workflow, el mensaje de error, el último nodo ejecutado y el enlace a la ejecución, que se usan para enviar una alerta por correo o Slack.

Un flujo con Schedule Trigger corre sin que nadie mire al editor. Si falla (por ejemplo, porque vence la credencial de Google Sheets o hay un corte de red), la única huella es una ejecución marcada en rojo en el listado de **Executions**: nadie recibe aviso y el síntoma es solo que el resumen diario no llega, algo que puede pasar días sin ser notado. Con un Error Workflow la falla genera una alerta inmediata. Dos precauciones: solo se activa en ejecuciones automáticas (no en pruebas manuales) y un nodo con **Continue** oculta el error a este mecanismo.

5.4 Casos de frontera (U = 74.000 y D = 7)

Las 15 solicitudes entregadas no incluyen casos de frontera, por lo que se construyeron dos. Ambos tienen estado *sin_abrir*, para que ninguna otra causal interfiera. Según la regla 6 de la política (comparaciones inclusivas), ambos deben quedar aprobados.

Frontera de monto (monto = U)

Carga enviada

```
{
  "id_solicitud": "DEV-F01",
  "sku": "PL-120",
```

Esperado	<pre>"monto": "74000", "dias_desde_compra": "3", "estado_producto": "sin_abrir", "email_cliente": "cliente.f01@example.cl" }</pre> La regla 5 ($\text{monto} > 74000$) no se cumple porque 74000 no es mayor que 74000; ninguna otra regla aplica → Fallback → ruta APROBACION.
------------------------	--

Frontera de plazo (días = D)

Carga enviada	<pre>{ "id_solicitud": "DEV-F02", "sku": "PL-120", "monto": "45990", "dias_desde_compra": "7", "estado_producto": "sin_abrir", "email_cliente": "cliente.f02@example.cl" }</pre>
Esperado	La regla 3 ($\text{dias_desde_compra} > 7$) no se cumple porque 7 no es mayor que 7 → Fallback → ruta APROBACION ("está en plazo").

5.5 Los dos casos que el flujo no puede resolver

Caso A — "monto": "veinte mil"

Carga enviada	<pre>{ "id_solicitud": "DEV-X01", "sku": "PL-120", "monto": "veinte mil", "dias_desde_compra": "2", "estado_producto": "sin_abrir", "email_cliente": "cliente.x01@example.cl" }</pre>
Qué ocurre	El Switch falla al evaluar "veinte mil".toNumber(): el texto no es convertible a número. Con <i>*Stop Workflow*</i> la ejecución se detiene, sin ruta, sin registro y sin respuesta útil para el emisor.
Por qué	El flujo supone que monto es un texto numérico; solo valida que sea mayor que cero, no que sea un número. La conversión se hace dentro del propio Switch, así que un valor no convertible rompe el nodo en vez de ser clasificado. El formulario real no impone el formato y el flujo no se defiende.
Qué necesitaría	Un nodo de validación previo al Switch (IF/Filter con expresión regular o comprobación de

número) que envíe estos casos a INVALIDO_MONTO, y un Error Workflow que alerte de cualquier falla residual.

Caso B — "sku": "ZZ-999"

Carga enviada	<pre>{ "id_solicitud": "DEV-X02", "sku": "ZZ-999", "monto": "45990", "dias_desde_compra": "2", "estado_producto": "sin_abrir", "email_cliente": "cliente.x02@example.cl" }</pre>
Qué ocurre	Ningún nodo falla: el SKU pasa la validación y la solicitud se clasifica como válida (APROBACION si no hay otra causal), aunque el producto no existe.
Por qué	Las reglas de la política solo evalúan id, monto, días y estado; el SKU se trata como texto libre. El flujo no conoce el catálogo de productos, por lo que no puede distinguir un SKU real de uno inventado. El resultado es una aprobación automática sin sustento, sin ningún aviso.
Qué necesitaría	Acceso al catálogo (hoja o base de datos) y un nodo de búsqueda por SKU (Google Sheets *Get row(s)* o Merge) seguido de una regla que envíe los SKU inexistentes a DATOS_INVALIDOS o a REVISION. Idealmente, también contrastar el monto con el precio del catálogo.